

Entregable Sprint 2

Apache Cassandra

Modelo de Familias de Columnas y Scripts CQL

para Tablas Especializadas y Consultas Gerenciales

Proyecto:	Plataforma Poliglota - Museo de Arte Contemporaneo
Curso:	Sistemas de Bases de Datos II
Motor:	Apache Cassandra 4.1 (Docker)
Paradigma:	Query-Driven Modeling (Familias de Columnas)

1. Objetivo del Sprint

Disenar familias de columnas basadas estrictamente en las consultas del administrador del museo, definiendo correctamente las Partition Keys y Clustering Columns para garantizar lecturas $O(1)$ y un modelo que siga el principio de Query-Driven Modeling.

El entregable incluye: el modelo de 5 familias de columnas con justificacion de cada Partition Key y Clustering Column, los scripts CQL de creacion (DDL), y las 10 consultas gerenciales que demuestran como cada tabla responde a una pregunta de negocio concreta.

2. Principio de Diseno: Query-Driven Modeling

En Apache Cassandra, el modelo de datos se disena a partir de las consultas que necesita el administrador, NO a partir de las entidades (como en el modelo relacional). Esto implica:

- 1) Se identifica cada pregunta del negocio.
- 2) Se crea UNA tabla dedicada a responder esa pregunta.
- 3) Se eligen Partition Key y Clustering Columns de forma que la consulta solo necesite acceder a UNA particion (lectura $O(1)$).
- 4) Se DESNORMALIZAN datos (se duplican entre tablas) para evitar JOINS, que Cassandra no soporta.

Trade-off aceptado: se usa mas espacio en disco a cambio de lecturas extremadamente rapidas y alta disponibilidad.

3. Definicion del Keyspace

```
CREATE KEYSPACE IF NOT EXISTS museo
WITH replication = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};
```

Se utiliza SimpleStrategy con factor de replicacion 1 para el entorno de desarrollo local (un unico nodo Docker). En un ambiente de produccion se utilizaria NetworkTopologyStrategy con RF=3 para garantizar tolerancia a fallos.

4. Modelo de Familias de Columnas

Se disenaron 5 familias de columnas. A continuacion se presenta cada una con su consulta asociada, justificacion de la Partition Key (PK) y las Clustering Columns (CC), y el script CQL de creacion.

4.1 Tabla: ventas_por_mes

Consulta que responde: Cuanto se facturo en el mes X del ano Y?

Partition Key: (year, month)

Clustering Columns: sale_date DESC, sale_id DESC

Justificacion de la PK y CC:

- La PK compuesta (year, month) agrupa todas las ventas de un mismo mes en una sola particion, permitiendo obtener el resumen mensual en una sola lectura.
- sale_date DESC como primera CC ordena las ventas cronologicamente (mas recientes primero).
- sale_id DESC como segunda CC desambigua ventas que ocurren en el mismo instante, evitando colisiones de clave.

Script CQL:

```
CREATE TABLE IF NOT EXISTS ventas_por_mes (  
  year          INT,  
  month         INT,  
  sale_date     TIMESTAMP,  
  sale_id       INT,  
  artwork_id   INT,  
  artwork_title TEXT,  
  artist_name   TEXT,  
  genre_name    TEXT,  
  buyer_username TEXT,  
  payment_method TEXT,  
  subtotal      DECIMAL,  
  iva           DECIMAL,  
  commission    DECIMAL,  
  total         DECIMAL,  
  PRIMARY KEY ((year, month), sale_date, sale_id)  
) WITH CLUSTERING ORDER BY (sale_date DESC, sale_id DESC);
```

Ejemplo de consulta CQL:

```
SELECT * FROM ventas_por_mes WHERE year = 2025 AND month = 6;
```

4.2 Tabla: ventas_por_artista

Consulta que responde: Cuales son las ventas del artista X?

Partition Key: (artist_name)

Clustering Columns: sale_date DESC, sale_id DESC

Justificacion de la PK y CC:

- La PK (artist_name) agrupa todas las ventas de un artista en una sola particion. Esto permite calcular la facturacion total, comisiones y ranking de un artista con una unica lectura.
- sale_date DESC ordena cronologicamente y sale_id desambigua.

Script CQL:

```
CREATE TABLE IF NOT EXISTS ventas_por_artista (  
  artist_name      TEXT,  
  sale_date        TIMESTAMP,  
  sale_id          INT,  
  artwork_id      INT,  
  artwork_title   TEXT,  
  genre_name       TEXT,  
  buyer_username  TEXT,  
  payment_method   TEXT,  
  subtotal         DECIMAL,  
  iva              DECIMAL,  
  commission       DECIMAL,  
  total            DECIMAL,  
  PRIMARY KEY ((artist_name), sale_date, sale_id)  
) WITH CLUSTERING ORDER BY (sale_date DESC, sale_id DESC);
```

Ejemplo de consulta CQL:

```
SELECT * FROM ventas_por_artista WHERE artist_name = 'Pablo Picasso';
```

4.3 Tabla: ventas_por_genero

Consulta que responde: Cuales son las ventas del genero artistico X?

Partition Key: (genre_name)

Clustering Columns: sale_date DESC, sale_id DESC

Justificacion de la PK y CC:

- La PK (genre_name) agrupa ventas por tipo de arte, permitiendo analisis de tendencias del mercado artistico con una sola lectura.
- Es una tabla separada de ventas_por_artista porque Cassandra no soporta consultas eficientes por columnas que no sean parte de la PK. Duplicar datos es el patron correcto.

Script CQL:

```
CREATE TABLE IF NOT EXISTS ventas_por_genero (  
    genre_name        TEXT,  
    sale_date         TIMESTAMP,  
    sale_id           INT,  
    artwork_id       INT,  
    artwork_title    TEXT,  
    artist_name       TEXT,  
    buyer_username    TEXT,  
    payment_method    TEXT,  
    subtotal          DECIMAL,  
    iva               DECIMAL,  
    commission        DECIMAL,  
    total             DECIMAL,  
    PRIMARY KEY ((genre_name), sale_date, sale_id)  
) WITH CLUSTERING ORDER BY (sale_date DESC, sale_id DESC);
```

Ejemplo de consulta CQL:

```
SELECT * FROM ventas_por_genero WHERE genre_name = 'Pintura';
```

Justificacion de 3 tablas de ventas: Cassandra no soporta JOINS ni indices secundarios eficientes. Tener ventas_por_mes, ventas_por_artista y ventas_por_genero es el patron correcto: se duplican datos para que cada consulta sea $O(1)$. El trade-off en espacio es aceptable dado el volumen manejable de un museo.

4.4 Tabla: bitacora_eventos (Auditoria)

Consulta que responde: Que eventos de tipo X ocurrieron en el mes Y?

Partition Key: (event_year, event_month, event_type)

Clustering Columns: event_timestamp DESC, event_id DESC

Justificacion de la PK y CC:

- La PK compuesta (event_year, event_month, event_type) evita particiones que crezcan infinitamente. Si solo se usara event_type, la particion 'VENTA' creceria sin limite. Al agregar year+month, cada particion esta ACOTADA (bounded partition) al numero de eventos de un tipo en un mes.
- event_id es UUID para evitar colisiones cuando dos eventos ocurren en el mismo timestamp exacto (patron estandar para logs en Cassandra).
- Esta tabla es IMMUTABLE: solo INSERT, nunca UPDATE ni DELETE.

Tipos de evento soportados: VENTA, RESERVA, CANCELACION, CAMBIO_ESTATUS, REGISTRO_USUARIO, MEMBRESIA.

Script CQL:

```
CREATE TABLE IF NOT EXISTS bitacora_eventos (  
    event_year          INT,  
    event_month         INT,  
    event_type          TEXT,  
    event_timestamp     TIMESTAMP,  
    event_id            UUID,  
    user_username       TEXT,  
    description         TEXT,  
    entity_type        TEXT,  
    entity_id          TEXT,  
    old_value          TEXT,  
    new_value          TEXT,  
    PRIMARY KEY ((event_year, event_month, event_type),  
                event_timestamp, event_id)  
) WITH CLUSTERING ORDER BY  
    (event_timestamp DESC, event_id DESC);
```

Ejemplo de consulta CQL:

```
SELECT * FROM bitacora_eventos  
WHERE event_year = 2025  
    AND event_month = 3  
    AND event_type = 'VENTA';
```

4.5 Tabla: historial_estatus_obra

Consulta que responde: Cual es el historial de cambios de la obra X?

Partition Key: (artwork_id)

Clustering Columns: change_timestamp DESC

Justificacion de la PK y CC:

- La PK (artwork_id) agrupa todo el historial de una obra en una sola particion. Cada obra tiene un numero limitado de cambios de estatus (tipicamente 2-4: AVAILABLE -> RESERVED -> SOLD), asi que la particion esta naturalmente acotada.
- change_timestamp DESC muestra los cambios mas recientes primero.
- Se desnormalizan artwork_title y changed_by para evitar consultas adicionales.

Script CQL:

```
CREATE TABLE IF NOT EXISTS historial_estatus_obra (  
  artwork_id      INT,  
  change_timestamp TIMESTAMP,  
  artwork_title   TEXT,  
  old_status      TEXT,  
  new_status      TEXT,  
  changed_by      TEXT,  
  reason          TEXT,  
  PRIMARY KEY ((artwork_id), change_timestamp)  
) WITH CLUSTERING ORDER BY (change_timestamp DESC);
```

Ejemplo de consulta CQL:

```
SELECT * FROM historial_estatus_obra WHERE artwork_id = 1;
```

5. Resumen de Partition Keys y Clustering Columns

Tabla	Partition Key	Clustering	Consulta
ventas_por_mes	(year, month)	sale_date, sale_id	Facturacion mensual
ventas_por_artista	(artist_name)	sale_date, sale_id	Ventas por artista
ventas_por_genero	(genre_name)	sale_date, sale_id	Ventas por genero
bitacora_eventos	(year, month, type)	timestamp, event_id	Auditoria por tipo/mes
historial_estatus	(artwork_id)	change_timestamp	Historial de obra

Todas las tablas utilizan CLUSTERING ORDER BY ... DESC para que los datos mas recientes aparezcan primero, evitando ORDER BY en tiempo de consulta (que Cassandra solo permite en el orden del clustering definido en la tabla).

6. Consultas Gerenciales Implementadas

Se implementaron 10 consultas gerenciales divididas en dos bloques: facturacion y auditoria. Cada consulta aprovecha el modelo Query-Driven para resolver la pregunta de negocio en una sola lectura por particion.

Bloque 1: Facturacion

Q1: Facturacion de un mes especifico

Tabla: *ventas_por_mes*

```
SELECT * FROM ventas_por_mes
WHERE year = ? AND month = ?;
```

Q2: Facturacion de un periodo (rango de meses)

Tabla: *ventas_por_mes*

```
-- Iteracion mes a mes en Python
SELECT subtotal, iva, commission, total
FROM ventas_por_mes
WHERE year = ? AND month = ?;
```

Q3: Ventas de un artista especifico

Tabla: *ventas_por_artista*

```
SELECT * FROM ventas_por_artista
WHERE artist_name = ?;
```

Q4: Ventas de un genero especifico

Tabla: *ventas_por_genero*

```
SELECT * FROM ventas_por_genero
WHERE genre_name = ?;
```

Q5: Ranking de artistas por facturacion

Tabla: *ventas_por_artista*

```
SELECT total, commission
FROM ventas_por_artista
WHERE artist_name = ?;
-- Iteracion por artista, ordenar en Python
```

Q6: Ranking de generos por facturacion

Tabla: *ventas_por_genero*


```
SELECT total FROM ventas_por_genero
WHERE genre_name = ?;
-- Iteracion por genero, ordenar en Python
```

Bloque 2: Auditoria y Bitacora

Q7: Eventos por tipo en un mes

Tabla: bitacora_eventos

```
SELECT * FROM bitacora_eventos
WHERE event_year = ?
      AND event_month = ?
      AND event_type = ?;
```

Q8: Resumen de todos los eventos del mes

Tabla: bitacora_eventos

```
-- Iteracion por cada event_type:
SELECT event_timestamp, user_username, description
FROM bitacora_eventos
WHERE event_year = ?
      AND event_month = ?
      AND event_type = ?;
```

Q9: Historial de estatus de una obra

Tabla: historial_estatus_obra

```
SELECT * FROM historial_estatus_obra
WHERE artwork_id = ?;
```

Q10: Listado de obras vendidas en periodo

Tabla: ventas_por_mes

```
SELECT sale_id, sale_date, artwork_title,
       artist_name, genre_name,
       buyer_username, total
FROM ventas_por_mes
WHERE year = ? AND month = ?;
-- Iteracion mes a mes del rango
```

7. Justificacion de Decisiones de Diseno

Por que 3 tablas de ventas en vez de 1?

Cassandra no soporta JOINS ni indices secundarios eficientes. Tener ventas_por_mes, ventas_por_artista y ventas_por_genero es el patron correcto en Cassandra: se duplican datos para que cada consulta sea O(1) por partition key. El trade-off (espacio extra) es aceptable dado que las ventas de un museo son un volumen manejable.

Por que la bitacora tiene (year, month, type) como PK?

Si usamos solo event_type, la particion de 'VENTA' creceria infinitamente. Al agregar year + month, cada particion esta ACOTADA (bounded partition) al numero de eventos de un tipo en un mes. Este es un patron recomendado para time-series en Cassandra.

Por que UUID en event_id?

Evita colisiones cuando dos eventos ocurren en el mismo timestamp exacto. Es el patron estandar para logs en Cassandra.

Por que Clustering Order DESC?

Los administradores siempre quieren ver primero lo mas reciente. DESC evita un ORDER BY en la consulta, que Cassandra ejecuta eficientemente SOLO en el orden del clustering definido en la tabla.

Coexistencia MySQL <-> Cassandra

MySQL (core relacional) es el origen de verdad transaccional (ACID). Cassandra es el almacen de lectura optimizado para reportes gerenciales. MySQL maneja consistencia fuerte; Cassandra ofrece consistencia eventual (aceptable para reportes historicos). Los datos fluyen de MySQL a Cassandra via el script ETL de migracion.